contains-studio / agents

Code | Issues 8 | Pull requests 4 | Discussions | Actions | Projects | Security | Insights

Watch 193 | Use this template

sharing current agents in use

12.2k stars | 2.5k forks | 193 watching | Branches | Activity | Custom properties | Tags

Public template repository

main | 1 Branch | 0 Tags | Go to file t | Go to file | Add file | Code

mgalpert Delete CLAUDE.md a5a480c · 7 months ago

bonus	Update all references from 6-week to 6-day sp...	7 months ago
design	Update all references from 6-week to 6-day sp...	7 months ago
engineering	Update all references from 6-week to 6-day sp...	7 months ago
marketing	Update all references from 6-week to 6-day sp...	7 months ago
product	Update all references from 6-week to 6-day sp...	7 months ago
project-management	Update all references from 6-week to 6-day sp...	7 months ago
studio-operations	Restructure repository for easy agent installati...	7 months ago
testing	Update all references from 6-week to 6-day sp...	7 months ago
.gitignore	Restructure repository for easy agent installati...	7 months ago
README.md	Update README.md	7 months ago

README

Contains Studio AI Agents

A comprehensive collection of specialized AI agents designed to accelerate and enhance every aspect of rapid development. Each agent is an expert in their domain, ready to be invoked when their expertise is needed.

📥 Installation

1. **Download this repository:**

```
git clone https://github.com/contains-studio/agents.git
```

2. **Copy to your Claude Code agents directory:**

```
cp -r agents/* ~/.claude/agents/
```

   Or manually copy all the agent files to your `~/.claude/agents/` directory.

3. **Restart Claude Code** to load the new agents.

🚀 Quick Start

Agents are automatically available in Claude Code. Simply describe your task and the appropriate agent will be triggered. You can also explicitly request an agent by mentioning their name.

📚 **Learn more:** [Claude Code Sub-Agents Documentation]

Example Usage

- "Create a new app for tracking meditation habits" → `rapid-prototyper`
- "What's trending on TikTok that we could build?" → `trend-researcher`
- "Our app reviews are dropping, what's wrong?" → `feedback-synthesizer`
- "Make this loading screen more fun" → `whimsy-injector`

📁 Directory Structure

Agents are organized by department for easy discovery:

```
contains-studio-agents/
├── design/
│   ├── brand-guardian.md
│   ├── ui-designer.md
│   ├── ux-researcher.md
│   ├── visual-storyteller.md
│   └── whimsy-injector.md
├── engineering/
│   ├── ai-engineer.md
│   ├── backend-architect.md
│   ├── devops-automator.md
│   ├── frontend-developer.md
│   ├── mobile-app-builder.md
│   ├── rapid-prototyper.md
│   └── test-writer-fixer.md
├── marketing/
│   ├── app-store-optimizer.md
│   ├── content-creator.md
│   ├── growth-hacker.md
│   ├── instagram-curator.md
│   ├── reddit-community-builder.md
│   ├── tiktok-strategist.md
│   └── twitter-engager.md
├── product/
│   ├── feedback-synthesizer.md
│   ├── sprint-prioritizer.md
│   └── trend-researcher.md
├── project-management/
│   ├── experiment-tracker.md
│   ├── project-shipper.md
│   └── studio-producer.md
├── studio-operations/
│   ├── analytics-reporter.md
│   ├── finance-tracker.md
│   ├── infrastructure-maintainer.md
│   ├── legal-compliance-checker.md
│   └── support-responder.md
├── testing/
│   ├── api-tester.md
│   ├── performance-benchmarker.md
│   ├── test-results-analyzer.md
│   ├── tool-evaluator.md
│   └── workflow-optimizer.md
└── bonus/
    ├── joker.md
    └── studio-coach.md
```

📋 Complete Agent List

Engineering Department (`engineering/`)

- **ai-engineer** - Integrate AI/ML features that actually ship
- **backend-architect** - Design scalable APIs and server systems
- **devops-automator** - Deploy continuously without breaking things
- **frontend-developer** - Build blazing-fast user interfaces
- **mobile-app-builder** - Create native iOS/Android experiences
- **rapid-prototyper** - Build MVPs in days, not weeks
- **test-writer-fixer** - Write tests that catch real bugs

Product Department (`product/`)

- **feedback-synthesizer** - Transform complaints into features
- **sprint-prioritizer** - Ship maximum value in 6 days
- **trend-researcher** - Identify viral opportunities

Marketing Department (`marketing/`)

- **app-store-optimizer** - Dominate app store search results
- **content-creator** - Generate content across all platforms
- **growth-hacker** - Find and exploit viral growth loops
- **instagram-curator** - Master the visual content game
- **reddit-community-builder** - Win Reddit without being banned
- **tiktok-strategist** - Create shareable marketing moments
- **twitter-engager** - Ride trends to viral engagement

Design Department (`design/`)

- **brand-guardian** - Keep visual identity consistent everywhere
- **ui-designer** - Design interfaces developers can actually build
- **ux-researcher** - Turn user insights into product improvements
- **visual-storyteller** - Create visuals that convert and share
- **whimsy-injector** - Add delight to every interaction

Project Management (`project-management/`)

- **experiment-tracker** - Data-driven feature validation
- **project-shipper** - Launch products that don't crash
- **studio-producer** - Keep teams shipping, not meeting

Studio Operations (`studio-operations/`)

- **analytics-reporter** - Turn data into actionable insights
- **finance-tracker** - Keep the studio profitable
- **infrastructure-maintainer** - Scale without breaking the bank
- **legal-compliance-checker** - Stay legal while moving fast
- **support-responder** - Turn angry users into advocates

Testing & Benchmarking (`testing/`)

- **api-tester** - Ensure APIs work under pressure
- **performance-benchmarker** - Make everything faster
- **test-results-analyzer** - Find patterns in test failures
- **tool-evaluator** - Choose tools that actually help

- **workflow-optimizer** - Eliminate workflow bottlenecks

🎁 Bonus Agents

- **studio-coach** - Rally the AI troops to excellence
- **joker** - Lighten the mood with tech humor

🎯 Proactive Agents

Some agents trigger automatically in specific contexts:

- **studio-coach** - When complex multi-agent tasks begin or agents need guidance
- **test-writer-fixer** - After implementing features, fixing bugs, or modifying code
- **whimsy-injector** - After UI/UX changes
- **experiment-tracker** - When feature flags are added

💡 Best Practices

1. **Let agents work together** - Many tasks benefit from multiple agents
2. **Be specific** - Clear task descriptions help agents perform better
3. **Trust the expertise** - Agents are designed for their specific domains
4. **Iterate quickly** - Agents support the 6-day sprint philosophy

🔧 Technical Details

Agent Structure

Each agent includes:

- **name**: Unique identifier
- **description**: When to use the agent with examples
- **color**: Visual identification
- **tools**: Specific tools the agent can access
- **System prompt**: Detailed expertise and instructions

Adding New Agents

1. Create a new `.md` file in the appropriate department folder
2. Follow the existing format with YAML frontmatter
3. Include 3-4 detailed usage examples
4. Write comprehensive system prompt (500+ words)
5. Test the agent with real tasks

📊 Agent Performance

Track agent effectiveness through:

- Task completion time
- User satisfaction
- Error rates
- Feature adoption
- Development velocity

🚦 Status

- ✅ **Active**: Fully functional and tested

- 🚧 **Coming Soon**: In development
- 🧪 **Beta**: Testing with limited functionality

🛠️ Customizing Agents for Your Studio

Agent Customization Todo List

Use this checklist when creating or modifying agents for your specific needs:

📋 Required Components

- [ ] **YAML Frontmatter**
  - [ ] `name` : Unique agent identifier (kebab-case)
  - [ ] `description` : When to use + 3-4 detailed examples with context/commentary
  - [ ] `color` : Visual identification (e.g., blue, green, purple, indigo)
  - [ ] `tools` : Specific tools the agent can access (Write, Read, MultiEdit, Bash, etc.)

📝 System Prompt Requirements (500+ words)

- [ ] **Agent Identity**: Clear role definition and expertise area
- [ ] **Core Responsibilities**: 5-8 specific primary duties
- [ ] **Domain Expertise**: Technical skills and knowledge areas
- [ ] **Studio Integration**: How agent fits into 6-day sprint workflow
- [ ] **Best Practices**: Specific methodologies and approaches
- [ ] **Constraints**: What the agent should/shouldn't do
- [ ] **Success Metrics**: How to measure agent effectiveness

🎯 Required Examples by Agent Type

Engineering Agents need examples for:

- [ ] Feature implementation requests
- [ ] Bug fixing scenarios
- [ ] Code refactoring tasks
- [ ] Architecture decisions

Design Agents need examples for:

- [ ] New UI component creation
- [ ] Design system work
- [ ] User experience problems
- [ ] Visual identity tasks

Marketing Agents need examples for:

- [ ] Campaign creation requests
- [ ] Platform-specific content needs
- [ ] Growth opportunity identification
- [ ] Brand positioning tasks

Product Agents need examples for:

- [ ] Feature prioritization decisions
- [ ] User feedback analysis
- [ ] Market research requests
- [ ] Strategic planning needs

Operations Agents need examples for:

- [ ] Process optimization

- [ ] Tool evaluation
- [ ] Resource management
- [ ] Performance analysis

✅ Testing & Validation Checklist

- [ ] **Trigger Testing**: Agent activates correctly for intended use cases
- [ ] **Tool Access**: Agent can use all specified tools properly
- [ ] **Output Quality**: Responses are helpful and actionable
- [ ] **Edge Cases**: Agent handles unexpected or complex scenarios
- [ ] **Integration**: Works well with other agents in multi-agent workflows
- [ ] **Performance**: Completes tasks within reasonable timeframes
- [ ] **Documentation**: Examples accurately reflect real usage patterns

🔧 Agent File Structure Template

```
---
name: your-agent-name
description: Use this agent when [scenario]. This agent specializes in [expertise]. Examples:\n\n<example>\nContext:
color: agent-color
tools: Tool1, Tool2, Tool3
---

You are a [role] who [primary function]. Your expertise spans [domains]. You understand that in 6-day sprints, [spri

Your primary responsibilities:
1. [Responsibility 1]
2. [Responsibility 2]
...

[Detailed system prompt content...]

Your goal is to [ultimate objective]. You [key behavior traits]. Remember: [key philosophy for 6-day sprints].
```

📁 Department-Specific Guidelines

Engineering (`engineering/`): Focus on implementation speed, code quality, testing **Design** (`design/`): Emphasize user experience, visual consistency, rapid iteration
Marketing (`marketing/`): Target viral potential, platform expertise, growth metrics **Product** (`product/`): Prioritize user value, data-driven decisions, market fit **Operations** (`studio-operations/`): Optimize processes, reduce friction, scale systems **Testing** (`testing/`): Ensure quality, find bottlenecks, validate performance **Project Management** (`project-management/`): Coordinate teams, ship on time, manage scope

🎨 Customizations

Modify these elements for your needs:

- [ ] Adjust examples to reflect your product types
- [ ] Add specific tools agents have access to
- [ ] Modify success metrics for your KPIs
- [ ] Update department structure if needed
- [ ] Customize agent colors for your brand

🤝 Contributing

To improve existing agents or suggest new ones:

1. Use the customization checklist above

Releases

No releases published

Packages

No packages published